时空道宇星座

基础通信 SDK 使用说明

V1.0

保密：一级

目录

1. 概述 3
2. 集成环境要求 3
3. 集成步骤 3
4. 对象解释 3
4.1. CustomError 3
4.2. TaskState 4
4.3. SubjectTaskBean 4
4.4. EntityICI 4
4.5. EntityDII 5
4.6. EntityMCI 6
4.7. EntitySFI 6
5. 初始化 SDK 6
6. 模组数据监听 8
6.1. MCI 接收消息监听 8
6.2. SFQ 发送结果监听 8
6.3. 选定卫星信号监听 8
6.4. 地球站跟踪监听 9
6.5. DII 卫星通信单元信息监听 9
6.6. ICI 设备 IC 信息监听 9
7. 功能设置 10
7.1. 清空 SFQ 消息 10
7.2. 设置 COP 10
7.3. 发送 GMO 10
7.4. 更新模组经纬度 10
8. 自定义任务 11
8.1. 固件升级 11
8.2. 存储发送消息（SFQ） 12
8.3. 即时发送消息（MCQ） 13
8.4. 查询卫星离线消息 14
8.5. 查询剩余 SFQ 消息 15
9. 线程与回调说明 15

保密：一级

1. 概述

基础通信 SDK 是面向客户应用开发者提供的设备通信开发组件，用于简化客户 APP 与卫星通信终端之间的通信开发工作。SDK 对底层通信数据封装及消息收发等能力进行统一封装，客户可通过 SDK 提供的标准接口完成通信状态获取、消息发送及消息接收等功能。

项目	说明
SDK 名称	protocollibrary.aar
SDK 版本	v1.0.0
功能	包装模组基础功能使用，方便快速开发
适用平台	Android

2. 集成环境要求

配置项	要求
三方库	Rxjava
JVM Target	11
Android SDK	24+

3. 集成步骤

1. 将 aar 文件复制到主工程的 app/libs 目录，并添加 aar 依赖。
2. 添加 Rxjava 依赖

```
implementation 'io.reactivex.rxjava3:rxjava:3.1.12'
```

4. 对象解释

4.1.CustomError

自定义任务的一些特殊的错误类型

```
public enum CustomError {
    //初始化错误，代表 SDK 还未可用或者正在进行模组升级
    INITIALIZE_ERROR,

    //输入参数解析错误，表示调用接口时，传入的参数不正确
    INPUT_ERROR,

    //任务被移除，重复调用
    REMOVE,
```

保密：一级


```
    //调用 MCQ 发送消息时候的特殊错误，表示发送完成，但是未收到 ACK 回值
    NO_ACK

}
```

4.2. TaskState

当前任务的状态

```
public enum TaskState {
    //任务初始化
    INIT,
    //任务进行中
    START,
    //任务完成，并且任务成功
    COMPLETE,
    //任务完成，但是任务失败
    FAIL
}
```

4.3. SubjectTaskBean

任务状态的回调包装，里面包含了当前任务的状态，可携带一个自定义对象

```
public class SubjectTaskBean {
    //当前任务状态
    public final TaskState state;
    //携带的自定义对象
    public final Object bundle;
```

4.4. EntityICI

ICI 语句对应的 Java 解析对象

```
public class EntityICI {

    //ID 号:卫星号码（目前兼容终端 ID）
    private final String id;

    //链路模式:1- DCS-D
    private int LinkMode;
```

保密：一级


```
    //组网方式,参考协议 ICI 语句字段 3
    private int networkingType;


    //服务类型:1-长接入，2-短接入
    private int serviceType;


    //通信方式,参考协议 ICI 语句字段 6
    private int communicationType;


    //卫星应答模式:0-无应答 1-有应答
    private int satelliteResponseMode;


    //终端发送消息长度,参考协议 ICI 语句字段 9
    private int msgLength;
```

4.5.EntityDII

DII 语句对应的 Java 解析对象

```
public class EntityDII {


    //链路模式:1：DCS-D
    private final int DTU_Type;

    //模组软件版本
    private String Module_SoftwareVersion;

    //模组硬件版本
    private String Module_HardwareVersion;
```

保密：一级


```
    //设备序列号
    private String deviceSerialNumber;

    //针对大客户制定的标准版本号
    private String userVersion;
```

4.6.EntityMCI

```
public class EntityMCI {

    //utc 时间
    private final String UTCTime;

    //发信方号码
    private final String cardId;

    /**
     * 原始通信数据
     * 如果发送的是 16 进制字符，那这个就是 16 进制字符串
     */
    private String hexStr;
```

4.7.EntitySFI

```
public class EntitySFI {

    //发送状态：2：完成该报文发送，4：报文发射失败
    private final int state;

    //发送的报文 id
    private final int id;
```

5. 初始化 SDK

1.实例化 DispatchManager 对象，帮保持对象的单例

```
DispatchManager dispatchManager = new DispatchManager(new ModuleOperation() {
            /**
```

保密：一级


```
             * SDK 初始化
             * @param isReady 表示 SDK 是否初始化完成
             *                    返回 true 时才能开始自定义任务，否则返回
@CutomError.INITIALIZE_ERROR
             */
            @Override
            public void initialization(boolean isReady) {}

            /**
             * 输出所有 SDK 接收到的标准语句，方便应用自行解析
             * @param pactWrap
             */
            @Override
            public void passthrough(String pactWrap) {}

            /**
             * SDK 需要发送给模组的数据，由应用自己实现
             * @param data 需要发送的数据
             */
            @Override
            public void send(byte[] data) {}

            /**
             * 模组上电，具体操作由应用自己实现
             */
            @Override
            public void powerOn() {}
            /**
             * 模组断电，具体操作由应用自己实现
             */
            @Override
            public void powerDown() {}
        });
```

2.将模组输出的数据通过 DispatchManager 对象的 receive(final byte[] data)方法传入

保密：一级

3.调用 DispatchManager 的 start()方法，开启循环解析

6. 模组数据监听

通过 DispatchManager 的 getDataManager()，获取模组数据管理类 ModuleDataManager

6.1.接收消息监听（MCI）

1. 设置 MCI 语句监听，每当收到 MCI 语句输出，不保存上一个状态，建议 start 之前就可以设置

```
Disposable disposable = dataManager.setMCIMessageListening(new Consumer<EntityMCI>() {
            @Override
            public void accept(EntityMCI entityMCI) {

            }
        });
```

2.取消监听 disposable.dispose();

6.2.发送结果监听（SFI）

1. 设置 SFI 语句监听，每当模组里有存储的 SFQ 消息发送后返回，不保存上一个状态，建议 start 之前就可以设置

```
Disposable disposable = dataManager.setSFIMessageListening(new Consumer<EntitySFI>() {
            @Override
            public void accept(EntitySFI entitySFI) {

            }
        });
```

2.取消监听 disposable.dispose();

6.3.选定卫星信号监听（PDI）

1. 设置当前模组选定的卫星信号监听，保存上一个状态

```
Disposable disposable = dataManager.setSelectedSatelliteListening
 (new Consumer<Pair<String, Float>>() {
            @Override
            public void accept(Pair<String, Float> stringFloatPair) {
```

保密：一级


```
                //卫星名称，未锁定时为空字符
                String name = stringFloatPair.first;
                //锁定卫星的载噪比
                float cnr = stringFloatPair.second;
            }
        });
```

2.取消监听 disposable.dispose();

6.4.地球站跟踪监听

1. 设置当前选定的卫星是否有地面站跟踪监听，保存上一个状态

```
Disposable disposable = dataManager.setEarthStationListening(new Consumer<Boolean>() {
            @Override
            public void accept(Boolean aBoolean) {

            }
        });
```

2.取消监听 disposable.dispose();

6.5.卫星通信单元信息监听（DII）

```
Disposable disposable = dataManager.setDIIListening(new Consumer<EntityDII>() {
            @Override
            public void accept(EntityDII entityDII) {
                if (entityDII != EntityDII.empty) {
                    //可用
                }
            }
        });
```

2.取消监听 disposable.dispose();

6.6.设备 IC 信息监听（ICI）

```
Disposable disposable = dataManager.setICIListening(new Consumer<EntityICI>() {
            @Override
            public void accept(EntityICI entityICI) {
                if (entityICI != EntityICI.empty) {
```

保密：一级


```
                    //可用
                }
            }
        });
```

2.取消监听 disposable.dispose();

7. 功能设置

7.1.清空 SFQ 消息

```
dataManager.clearSFQ();
```

7.2.设置 COP

```
//传入要设置的波特率
dataManager.setCOP(int b);
```

7.3.发送 GMO

```
/**
 * 设定输出或停止输出参数语句，具体参数值参考 GMO 协议
 * @param target 目标语句
 * @param openPattern 打开模式
 * @param frequency 目标语句输出频度
 */
dataManager.sendGMO(String target, int openPattern, int frequency);
```

7.4.更新模组经纬度

```
/**
 * 更新模组经纬度，SDK 会保存最新的一次经纬度，方便在每次模组启动后自动设置，所以至少更新一次
 * 后续更新频率，参考协议 TPI 解释
 * @param longitude 经度，-180（西） ~ 180（东）
 * @param latitude 纬度，-90（南） ~ 90（北）
 */
dataManager.updateLocation (double longitude, double latitude);
```

保密：一级

8. 自定义任务

8.1. 固件升级

```
/**
 * 升级任务
 * @param context 上下文
 * @param src 升级模组程序文件
 * @param consumer 回调监听
 */
SubjectTask<?> subjectTask = dataManager.updateSOC(Context, src, new Consumer<SubjectTaskBean>() {
            //子线程
            @Override
            public void accept(SubjectTaskBean subjectTaskBean) {

                //state: TaskState.START 表示升级开始
                if (subjectTaskBean.state == TaskState.START) {
                    //progress: 升级进度 0~99
                    int progress = (int) Objects.requireNonNull(subjectTaskBean.bundle);

                }

                //state: TaskState.FAIL 表示任务结束，但是任务失败
                else if (subjectTaskBean.state == TaskState.FAIL) {

                    //失败原因：SDK 运行错误，参考对象解释@CustomError
                    if (subjectTaskBean.bundle instanceof CustomError) {

                    }
                    //失败原因：模组返回错误，参考协议升级文档定义
                    else if (subjectTaskBean.bundle instanceof Integer) {

                    }
```

保密：一级


```
                }

                //state: TaskState.COMPLETE 表示任务成功完成
                else if (subjectTaskBean.state == TaskState.COMPLETE) {

                }
            }
        });
```

8.2.存储发送消息（SFQ）

```
/**
 * 发送 SFQ 消息
 * @param cardId 卡号
 * @param content 16 进制字符串
 *                将要发送的 byte 转成 2 位 16 进制字符，不够补 0
 * @param consumer 回调
 */

SubjectTask<?> subjectTask = dataManager.sendMsgSFQ(cardId, content, new
Consumer<SubjectTaskBean>() {
            @Override
            public void accept(SubjectTaskBean subjectTaskBean) {

                //state: TaskState.FAIL 表示任务结束，但是任务失败
                if (subjectTaskBean.state == TaskState.FAIL) {

                    //失败原因：SDK 运行错误，参考对象解释@CustomError
                    if (subjectTaskBean.bundle instanceof CustomError) {

                    }
                    //失败原因：模组返回错误，参考文档 SFI 定义
                    else if (subjectTaskBean.bundle instanceof Integer) {

                    }
                }
```

保密：一级


```
                //state: TaskState.COMPLETE 表示任务成功完成
                else if (subjectTaskBean.state == TaskState.COMPLETE) {
                    //返回对应的报文 ID，方便后续通过 setSFIMessageListening 获取该条消息的发送状态
                    int id = (int) Objects.requireNonNull(subjectTaskBean.bundle);
                }
            }
        });
```

8.3.即时发送消息（MCQ）

```
SubjectTask<?> subjectTask = dataManager.sendMsgMCQ(cardId, content, new Consumer<SubjectTaskBean>() {
            @Override
            public void accept(SubjectTaskBean subjectTaskBean) {

                //state: TaskState.FAIL 表示任务结束，但是任务失败
                if (subjectTaskBean.state == TaskState.FAIL) {

                    //失败原因：SDK 运行错误，参考对象解释@CustomError
                    if (subjectTaskBean.bundle instanceof CustomError) {

                    }
                    //失败原因：模组返回错误，参考文档 FBI 定义
                    else if (subjectTaskBean.bundle instanceof Integer) {

                    }
                }

                //state: TaskState.COMPLETE 表示任务成功完成
                else if (subjectTaskBean.state == TaskState.COMPLETE) {
                  //表示发送完成，但是未收到 ACK
                  if (subjectTaskBean.bundle == CustomError. NO_ACK) {
```

保密：一级


```
}
}
        });
```

可通过 subjectTask.cancel(Object obj)取消发送

8.4.查询卫星离线消息

只有短接入终端具备该功能，短接入终端需要终端或者用户主动查询以接收下行消息

```
/**
 * 循环查询卫星剩余消息，直到无剩余消息，或查询失败
 * @param consumer 回调
 */

SubjectTask<?>        subjectTask        =        dataManager.queryOfflineMsg(new
Consumer<SubjectTaskBean>() {
    @Override
    public void accept(SubjectTaskBean subjectTaskBean) {

        //state: TaskState.START 表示任务正在进行
        if (subjectTaskBean.state == TaskState.START) {
            //卫星剩余消息数目,同时可能会通过 setMCIMessageListening 输出一条 MCI
            int remaining = (int) Objects.requireNonNull(subjectTaskBean.bundle);
        }

        //state: TaskState.FAIL 表示任务结束，但是任务失败
        else if (subjectTaskBean.state == TaskState.FAIL) {

            //失败原因：SDK 运行错误，参考对象解释@CustomError
            if (subjectTaskBean.bundle instanceof CustomError) {

            }
            //失败原因：模组返回错误，参考文档 CEI、CES 定义
            else if (subjectTaskBean.bundle instanceof Integer) {

            }
```

保密：一级


```
        }

        //state: TaskState.COMPLETE 表示任务成功完成
        else if (subjectTaskBean.state == TaskState.COMPLETE) {
        }
    }
});
```

可通过 subjectTask.cancel(Object obj)，中断查询

8.5.查询剩余 SFQ 消息

```
SubjectTask<?> subjectTask = dataManager.queryMsgSFQ(new Consumer<Integer>() {
    @Override
    public void accept(Integer integer) {
        if (integer < 0) {
            //查询失败
        }else {
            //查询成功，剩余 integer 条
        }
    }
});
```

可通过 subjectTask.cancel(Object obj)，中断查询

9. 线程与回调说明

1.所有回调均在子线程中，注意主线程切换

2.如果当前正在更新模组任务，所有数据发送和自定义任务都不可用